

Python Programming

Day 1: Python 기초

출력, 변수와 객체, 자료형, 연산자, 입력과 형 변환

Contents

1	학습 목표	2
2	Python과 첫 번째 프로그램	2
3	출력: print()	2
3.1	계산과 출력의 차이	3
3.2	문자열 출력	3
4	변수와 객체	3
4.1	대입 연산자 =	3
4.2	수학적 관점	4
5	같은 객체를 참조하는 경우	4
6	Mutable과 Immutable	5
6.1	Immutable 객체	5
6.2	Mutable 객체	5
7	기본 자료형	6
7.1	같은 값처럼 보여도 다른 자료형	6
8	부동소수점 오차	6
8.1	수학적 모델	7
8.2	연산 과정의 반올림	7
8.3	실수 비교	7
9	산술 연산자	7
9.1	일반 나눗셈	8
9.2	바닥 나눗셈	8
9.3	나머지	8
9.4	거듭제곱	8
10	입력: input()	8
10.1	왜 문자열로 입력되는가	9
10.2	형 변환	9
10.3	자료형 불일치	9
11	미니 실습: BMI 계산기	9
12	핵심 요약	10

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- `print()`를 이용한 출력
- 변수와 객체의 관계
- 대입 연산자 `=`의 의미
- Mutable 객체와 Immutable 객체의 차이
- 기본 자료형 `int`, `float`, `str`, `bool`
- 부동소수점 오차가 발생하는 이유
- 산술 연산자 `/`, `//`, `%`, `**`
- `input()`과 형 변환
- BMI 계산기 작성

2. Python과 첫 번째 프로그램

Python은 문법이 간결하고 읽기 쉬운 고급 프로그래밍 언어이다. 과학 계산, 데이터 분석, 인공지능, 자동화, 웹 개발 등 다양한 분야에서 사용된다.

물리학과 공학에서는 특히 다음 라이브러리와 함께 자주 사용된다.

- NumPy: 배열과 수치 계산
- SciPy: 과학 계산
- Matplotlib: 데이터 시각화
- Pandas: 표 형식 데이터 처리

가장 기본적인 Python 프로그램은 다음과 같다.

```
1 print("Hello, World!")
```

출력 결과:

Hello, World!

핵심 포인트

Python은 문법을 외우는 것보다, 코드가 왜 그렇게 동작하는지 이해하는 것이 중요하다.

3. 출력: `print()`

`print()`는 값을 화면에 출력하는 함수이다.

```
1 print("Python")
2 print(3 + 5)
3 print(10 * 20)
```

출력 결과:

```
Python
8
200
```

3.1. 계산과 출력의 차이

다음 코드는 계산은 수행하지만 일반적인 Python 스크립트에서는 결과를 화면에 보여 주지 않는다.

```
1 3 + 5
```

결과를 확인하려면 다음과 같이 `print()`를 사용해야 한다.

```
1 print(3 + 5)
```

핵심 포인트

계산(computation)과 출력(output)은 서로 다른 작업이다. `print()`는 계산 결과를 사람이 볼 수 있도록 화면에 전달한다.

3.2. 문자열 출력

문자열은 따옴표로 감싼다.

```
1 print("Hello")
2 print('Physics')
```

따옴표 없이 문자를 작성하면 Python은 그것을 변수 이름으로 해석하려고 한다.

```
1 print(Hello)
```

위 코드에서 `Hello`라는 이름이 정의되어 있지 않다면 오류가 발생한다.

4. 변수와 객체

다음 코드를 보자.

```
1 age = 22
```

입문 단계에서는 변수를 “값을 저장하는 상자”라고 설명하기도 한다. 하지만 Python에서는 변수를 객체를 가리키는 이름으로 이해하는 편이 더 정확하다.

`age` → 22

여기서 `age`는 숫자 자체가 아니라 정수 객체 22를 가리키는 이름이다.

4.1. 대입 연산자 =

다음 코드를 생각해 보자.

```
1 age = 22
2 age = 23
```

두 번째 줄은 22를 23으로 수정하는 것이 아니다.

1. 정수 객체 22가 존재한다.
2. `age`가 22를 가리킨다.

3. 정수 객체 23이 존재한다.
4. age가 23을 가리키도록 바뀐다.

즉, 바뀐 것은 객체가 아니라 age가 가리키는 대상이다.

핵심 포인트

Python의 =는 수학의 등호가 아니라 대입 연산자이다. $x = 3$ 은 “ x 와 3이 같다”가 아니라 “ x 가 정수 객체 3을 가리키게 하라”는 뜻이다.

4.2. 수학적 관점

변수들의 집합을 V , 객체들의 집합을 O 라고 하면 프로그램의 상태를 다음과 같은 함수로 생각할 수 있다.

$$f: V \rightarrow O$$

예를 들어 처음에는

$$f(\text{age}) = 22$$

였지만, 새로운 대입 이후에는

$$f(\text{age}) = 23$$

이 된다.

5. 같은 객체를 참조하는 경우

다음 코드를 보자.

```
1 a = 10
2 b = a
3
4 a = 20
5
6 print(a)
7 print(b)
```

출력 결과:

```
20
10
```

처음에는 a와 b가 모두 정수 객체 10을 가리킨다. 그러나 a = 20을 실행하면 a만 새로운 정수 객체 20을 가리키게 된다.

반면 다음 코드를 보자.

```
1 a = [1, 2, 3]
2 b = a
3
4 a.append(4)
5
6 print(a)
7 print(b)
```

출력 결과:

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

a와 b가 같은 리스트 객체를 가리키고 있고, `append()`가 리스트 객체 자체를 수정했기 때문에 두 이름을 통해 동일한 변화가 보인다.

6. Mutable과 Immutable

Python 객체는 객체 자체를 수정할 수 있는지에 따라 Mutable과 Immutable로 나눌 수 있다.

6.1. Immutable 객체

Immutable 객체는 객체 자체를 수정할 수 없다.

대표적인 예:

- `int`
- `float`
- `bool`
- `str`
- `tuple`

다음 코드는 정수 객체 10을 수정하지 않는다.

```
1 a = 10
2 a = 20
```

a가 새로운 정수 객체 20을 가리키게 될 뿐이다.

6.2. Mutable 객체

Mutable 객체는 내부 상태를 직접 수정할 수 있다.

대표적인 예:

- `list`
- `dict`
- `set`

```
1 numbers = [1, 2, 3]
2 numbers.append(4)
3
4 print(numbers)
```

출력 결과:

```
[1, 2, 3, 4]
```

체크 질문

다음 코드의 출력 결과를 예측해 보자.

```
1 a = [1, 2, 3]
2 b = a
3 b = [7, 8, 9]
4
5 print(a)
6 print(b)
```

b = [7, 8, 9]는 기존 리스트를 수정하는 것이 아니라 b가 새로운 리스트 객체를 가리키게 한다.

7. 기본 자료형

Python에서 자주 사용하는 기본 자료형은 다음과 같다.

자료형	의미	예시
int	정수	10
float	부동소수점 수	3.14
str	문자열	"Python"
bool	참/거짓	True

자료형은 객체가 어떤 연산을 수행할 수 있는지 결정한다.

```
1 a = 10
2 b = 3.14
3 c = "Physics"
4 d = True
5
6 print(type(a))
7 print(type(b))
8 print(type(c))
9 print(type(d))
```

7.1. 같은 값처럼 보여도 다른 자료형

```
1 a = 10
2 b = 10.0
3
4 print(type(a))
5 print(type(b))
```

10은 int, 10.0은 float이다. 수학적으로는 같은 값이지만 컴퓨터 내부의 표현 방식은 다르다.

8. 부동소수점 오차

다음 코드를 실행해 보자.

```
1 print(0.1 + 0.2)
```

출력 결과:

0.30000000000000004

이것은 Python의 버그가 아니다.

컴퓨터는 이진법을 사용하며, 십진수 0.1은 이진법에서 유한하게 표현되지 않는다. 따라서 컴퓨터는 표현 가능한 가장 가까운 값을 저장한다.

8.1. 수학적 모델

실수 전체를 $\mathbb{R}$, 컴퓨터가 표현 가능한 부동소수점 수의 집합을 F 라고 하자.

그러면 저장 과정을 다음 함수로 생각할 수 있다.

$$r : \mathbb{R} \rightarrow F$$

컴퓨터는 실제 값 x 대신 가장 가까운 표현 가능한 값 $r(x)$ 를 저장한다.

8.2. 연산 과정의 반올림

부동소수점 계산은 다음과 같이 생각할 수 있다.

$$\text{fl}(x + y) = r(r(x) + r(y))$$

입력값이 저장될 때 이미 반올림이 발생하고, 연산 결과 역시 표현 가능한 가장 가까운 값으로 다시 반올림된다.

8.3. 실수 비교

다음 두 비교는 서로 다른 결과를 낸다.

```
1 print(0.1 * 10 == 1.0)
2 print(0.1 * 3 == 0.3)
```

출력 결과:

```
True
False
```

과학 계산에서는 실수를 정확히 같은지 비교하기보다 허용 오차를 사용한다.

$$|a - b| < \varepsilon$$

핵심 포인트

부동소수점 오차는 유한한 비트로 실수를 표현할 때 발생하는 정상적인 현상이다.

9. 산술 연산자

연산자	의미	예시
+	덧셈	$a + b$
-	뺄셈	$a - b$
*	곱셈	$a * b$
/	나눗셈	a / b
//	바닥 나눗셈	$a // b$
%	나머지	$a \% b$
**	거듭제곱	$a ** b$

9.1. 일반 나눗셈

```
1 print(10 / 3)
```

출력 결과:

3.3333333333333335

9.2. 바닥 나눗셈

```
1 print(10 // 3)
2 print(-10 // 3)
```

출력 결과:

3
-4

Python의 //는 단순히 소수점 이하를 버리는 연산이 아니라 바닥 함수이다.

$$a//b = \left\lfloor \frac{a}{b} \right\rfloor$$

9.3. 나머지

Python의 //와 %는 다음 관계를 만족한다.

$$a = b(a//b) + (a\%b)$$

예를 들어

$$-10 = 3(-4) + 2$$

이므로

$$-10//3 = -4, \quad -10\%3 = 2$$

이다.

9.4. 거듭제곱

```
1 print(2 ** 3)
2 print(5 ** 2)
```

출력 결과:

8
25

10. 입력: input()

사용자로부터 값을 입력받기 위해 input()을 사용한다.

```
1 name = input("Name: ")
2 print(name)
```

input()은 항상 문자열을 반환한다.

```
1 age = input("Age: ")
2 print(type(age))
```

사용자가 20을 입력해도 자료형은 str이다.

10.1. 왜 문자열로 입력되는가

키보드 입력은 숫자와 문자를 자동으로 구분하지 않는다. 사용자가 입력한 내용을 먼저 문자들의 나열로 받아들이고, 그 이후 어떤 자료형으로 사용할지는 프로그래머가 결정한다.

10.2. 형 변환

정수로 변환하려면 `int()`를 사용한다.

```
1 age = int(input("Age: "))
```

실수로 변환하려면 `float()`를 사용한다.

```
1 height = float(input("Height: "))
```

수학적으로는 다음과 같은 함수 합성으로 생각할 수 있다.

$$K \xrightarrow{\text{input}} \text{String} \xrightarrow{\text{int}} \text{Integer}$$

10.3. 자료형 불일치

다음 코드는 오류를 발생시킨다.

```
1 age = input("Age: ")
2 print(age + 10)
```

`age`는 문자열이고 `10`은 정수이므로 두 자료형을 직접 더할 수 없다.
숫자 계산을 원한다면 다음과 같이 변환해야 한다.

```
1 age = int(input("Age: "))
2 print(age + 10)
```

11. 미니 실습: BMI 계산기

BMI는 다음 식으로 정의된다.

$$\text{BMI} = \frac{\text{weight}}{\text{height}^2}$$

키와 몸무게를 입력받아 BMI를 계산해 보자.

```
1 height = float(input("Enter your height (m): "))
2 weight = float(input("Enter your weight (kg): "))
3
4 bmi = weight / (height ** 2)
5
6 print(bmi)
```

예시 실행:

```
Enter your height (m): 1.80
Enter your weight (kg): 75
23.148148148148145
```

실습

이 프로그램은 다음 개념을 모두 사용한다.

- 변수
- 사용자 입력
- 형 변환
- 나눗셈
- 거듭제곱
- 출력

12. 핵심 요약

1. `print()`는 값을 화면에 출력한다.
2. 계산과 출력은 서로 다른 작업이다.
3. Python의 변수는 객체를 가리키는 이름이다.
4. `=`는 수학의 등호가 아니라 대입 연산자이다.
5. Mutable 객체는 내부 상태를 수정할 수 있다.
6. Immutable 객체는 내부 상태를 수정할 수 없다.
7. `int`, `float`, `str`, `bool`은 기본 자료형이다.
8. 부동소수점 오차는 실수를 유한한 비트로 표현하기 때문에 발생한다.
9. `//`는 바닥 나눗셈이고, `%`는 나머지 연산이다.
10. `input()`은 항상 문자열을 반환한다.
11. 숫자 계산에는 `int()` 또는 `float()`를 이용한 형 변환이 필요하다.